

Prototyping van een AI-geoptimaliseerde en Docker-gestuurde CI/CD-pipeline

Project: Continuous Integration & Continuous Delivery (CI/CD) voor het Voertuig evaluatieplatform

Groepje: Bram & Chris

Dit document beschrijft het volledige traject van de analyse, het ontwerp, de implementatie en de evaluatie van een CI/CD-pipeline voor het Voertuig Evaluatieplatform. Hierbij is de transitie gemaakt van een pure CI-omgeving naar een Continuous Delivery (CD) infrastructuur door middel van Docker-containerisatie en geautomatiseerde webhooks naar een Coolify-server, ondersteund door een AI-gestuurde foutanalyse loop via Gemini AI.

Inhoudsopgave

1. Analysefase	2
Huidige processen en AI-mogelijkheden	2
Identificatie van Knelpunten	2
De Rol van Virtualisatie en AI-optimalisatie	2
2. Ontwerpfase & Conceptuele Architectuur	3
Conceptuele Architectuur & Workflow	3
CI/CD Architectuurdiagram	4
Alternatieve Oplossingsrichtingen en Onderbouwing	5
3. Implementatie en Simulatiefase (MVP)	6
Technische Realisatie van het CD & AI component	6
Simulatie Scenario's en Resultaten	7
Ai Analysis Failure	8
4. Evaluatie, Reflectie en Schaalbaarheid	11
Prestatieanalyse van het AI- en Virtualisatie-ontwerp	11
Verbeterpunten en Toekomstvisie (Schaalbaarheid)	11

1. Analysefase

Huidige processen en AI-mogelijkheden

Binnen de ontwikkelomgeving van het Voertuig Evaluatieplatform werd er gebruikgemaakt van een simpele Continuous Integration (CI) structuur. Hoewel deze structuur effectief was voor het valideren van de codekwaliteit, vertoont dit proces tekortkomingen op het gebied van deployment (CD) en foutafhandeling.

Identificatie van Knelpunten

Tijdens de analyse van de bestaande werkwijze zijn de volgende cruciale knelpunten geïdentificeerd:

- **Knelpunt 1: Gebrek aan Continuous Delivery (CD):** Na een succesvolle build op de main-branch bleven de gebouwde frontend-artifacts (dist-map) binnen de GitHub-omgeving opgeslagen als build-artifact. Verschillende handmatige handelingen waren vereist om de code op de server te krijgen, wat leidde tot een trage release.
- **Knelpunt 2: Inefficiënte en reactieve foutdiagnose:** Wanneer tests faalden (bijvoorbeeld in de backend- of frontend-testfase), moesten developers handmatig de omvangrijke, logbestanden van GitHub Actions doorspitten om de root cause te achterhalen. Dit kostte veel tijd, nu hebben we daar een AI gestuurde implementatie voor gemaakt die automatisch feedback kan geven op Pull Requests en verbeteringen kan voorstellen.

De Rol van Virtualisatie en AI-optimalisatie

Om deze knelpunten op te lossen, is gekozen voor een driedelige strategie:

Github Actions

Voor de implementatie van de CI/CD-pipeline is gekozen voor **GitHub Actions**. De belangrijkste reden hiervoor was dat dit de eerste keer was dat een CI/CD-oplossing binnen het project werd opgezet.

Omdat de broncode al werd beheerd via GitHub, bood GitHub Actions een laagdrempelige en toegankelijke oplossing die direct geïntegreerd is binnen het platform. Hierdoor was geen extra infrastructuur of complexe configuratie nodig, wat de implementatie eenvoudiger maakte en de leercurve beperkte. Daarnaast maakte de nauwe integratie met het bestaande GitHub-project het mogelijk om snel geautomatiseerde workflows voor bouwen, testen en deployen op te

zetten.

Virtualisatie via Docker & Coolify

Door de applicatie te containeriseren via een Dockerfile, wordt de exacte runtime-omgeving gegarandeerd. Coolify is hierin een self-hosted PaaS (Platform-as-a-Service) die, vergelijkbaar met (en samen met) Docker Compose, de infrastructuur beheert.

Door een webhook te triggeren bij elke succesvolle merge op main, wordt de server getriggerd om de nieuwste container te bouwen en direct live te zetten. Dit sluit de CD-keten volledig.

AI-gestuurde Failure Detection & Debugging

Door integratie van de Gemini AI API (via het model Gemini-3.5-flash) worden logbestanden bij een falende pipeline direct asynchroon geanalyseerd. De AI bepaaldt de exacte fouten, de getroffen componenten en levert direct een codefix/ hint aan in de vorm van een GitHub Pull Request-comment of commit-notitie. Dit transformeert de feedbackloop van reactief naar proactief.

2. Ontwerpfase & Conceptuele Architectuur

Het ontwerp van het prototype is opgebouwd uit twee complementaire workflows binnen GitHub Actions: de primaire delivery pipeline en de secundaire AI-analyse pipeline.

Conceptuele Architectuur & Workflow

De architectuur is zo ontworpen dat taken strikt gescheiden en geoptimaliseerd zijn uitgevoerd:

Validatiefase

Parallele executie van test-backend en test-frontend op een Ubuntu runner om overhead te minimaliseren.

Build & CD-fase

Zodra de tests slagen en de code wordt samengevoegd met de main-branch, bouwt de pipeline

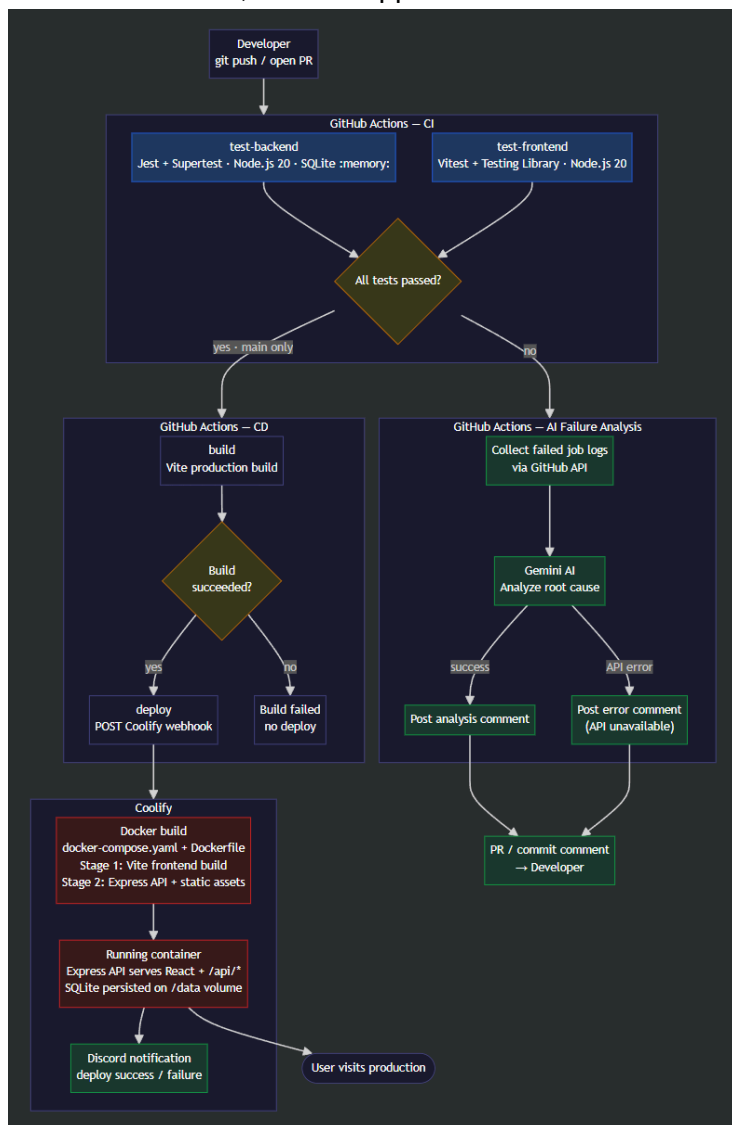
een applicatie als validatie, indien succesvol. Stuurt de pipeline een beveiligde POST-request (webhook) naar Coolify. Coolify pikt dit signaal op, leest de Docker-compose instellingen uit en start de geüpdatete container op de server met de nieuwste versie.

AI-feedback loop

Bij een onverhoopte fout treedt de workflow_run-trigger in werking via een aparte workflow. Deze opereert onafhankelijk, downloadt de logs van de gefaalde jobs via de GitHub REST API, kort deze in tot de laatste 6000 karakters om token-limits te bewaken, en raadpleegt Gemini AI.

CI/CD Architectuurdiagram

Hieronder weergegeven is ons CI/CD architectuurdiagram, deze beschrijft hoe we de pipeline en acties inrichten, welke stappen er moeten worden ondernomen en wat we gaan bouwen.



Alternatieve Oplossingsrichtingen en Onderbouwing

Technologiecomponent	Overwogen Alternatief	Gekozen Oplossing	Onderbouwing / Rechtvaardiging
Deployment Orchestrator	Kubernetes (K8s)	Docker + Coolify	Kubernetes introduceert te veel overhead en complexiteit

Technologiecomponent	Overwogen Alternatief	Gekozen Oplossing	Onderbouwing / Rechtvaardiging
		Webhook	voor een MVP van een voertuigevaluatieplatform. Coolify biedt de voordelen van Docker Compose-beheer via simpele webhooks zonder clusterbeheer-overhead.
AI-Foutanalyse Engine	Eigen getraind ML-model	Gemini-3.5-Flash via API	Een eigen model vereist enorme datasets aan logs en computationele resources. Gemini-3.5-Flash biedt out-of-the-box uitstekend begrip van codecontext, lage latentie en een kosteneffectieve API-structuur.

3. Implementatie en Simulatiefase (MVP)

Het prototype is volledig functioneel geïmplementeerd binnen de GitHub-repository van een namaakplatform. Hieronder vallen de workflows pipeline.yml, ai-analysis.yml en het executiescript analyze-failure.js.

Technische Realisatie van het CD & AI component

De uitgebreide CD-stap binnen de hoofd-pipeline ziet er conceptueel als volgt uit in de GitHub Actions-configuratie (toegevoegd onder de build job bij een merge op main):

```
- name: Trigger Coolify Webhook Deployment
  if: github.ref == 'refs/heads/main'
  run: |
    curl -X POST "${{ secrets.COOLIFY_WEBHOOK_URL }}" \
      -H "Authorization: Bearer ${{ secrets.COOLIFY_TOKEN }}" \
      -H "Content-Type: application/json"
```

Voor de AI-component verwerkt het Node.js-script `analyze-failure.js` de logbestanden async met een HTTPS POST-request naar de Gemini API endpoint:

```
const requestBody = JSON.stringify({
  contents: [{ parts: [{ text: prompt }] }],
  generationConfig: { maxOutputTokens: 1024 }
});
const path = `v1beta/models/gemini-3.5-flash:generateContent?key=${apiKey}`;
```

Simulatie Scenario's en Resultaten

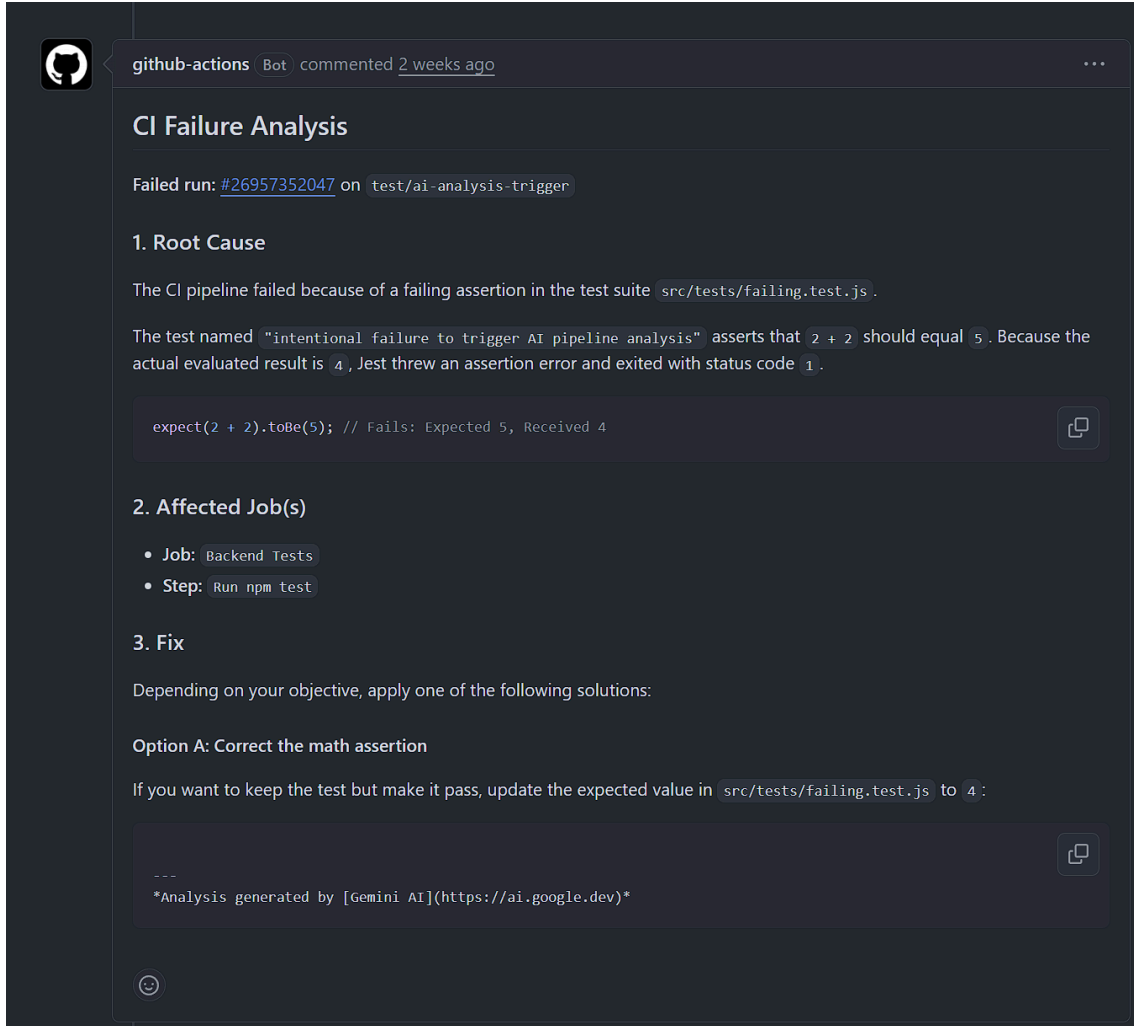
Om de werking en efficiëntie van het prototype te valideren, zijn er twee simulatie scenario's herhaaldelijk uitgevoerd:

Simulatiescenario	Geïntroduceerde conditie	Geobserveerd Gedrag Pipeline	Gemeten Doorlooptijd / Impact
1. Happy Path Deployment	Succesvolle code-wijziging in voertuig-validatielog ica gemerged naar main.	Backend/frontend-te sts slagen. Build voltooit succesvol. Artifact wordt geüpload en de Coolify webhook activeert direct de serverrebuild.	Totale pipeline: 2m 15s. Server deployment: 45s. Geen handmatige actie vereist.

Simulatiescenario	Geïntroduceerde conditie	Geobserveerd Gedrag Pipeline	Gemeten Doorlooptijd / Impact
2. Gefeilde Test met AI-analyse	Syntaxfout en foute assertie geïntroduceerd in de frontend-component en (client).	test-frontend faalt. De hoofd-pipeline stopt direct. De AI Pipeline Failure Analysis-workflow start direct na beëindiging, verzamelt de mislukte logs en post binnen no-time een heldere analyse als PR-comment.	AI-foutanalyse voltooid binnen 12 seconden na falen. Root cause accuraat gedefinieerd inclusief coderegel.

Ai Analysis Success

Wanneer we wel met een AI Model kunnen communiceren, plaatst hij op basis van de contents van de PR en de failing test met een mogelijke oplossing.



The screenshot shows a GitHub Actions comment from the 'github-actions' bot, posted 2 weeks ago. The comment is titled 'CI Failure Analysis' and provides a detailed analysis of a failed CI run. It identifies the root cause as a failing assertion in the test suite 'src/tests/failing.test.js', where a test named 'intentional failure to trigger AI pipeline analysis' failed because the actual result (4) did not match the expected result (5). The comment also lists the affected job ('Backend Tests') and step ('Run npm test'). Finally, it offers a fix: correcting the math assertion by updating the expected value from 5 to 4. The analysis is attributed to Gemini AI.

github-actions Bot commented 2 weeks ago

CI Failure Analysis

Failed run: [#26957352047](#) on `test/ai-analysis-trigger`

1. Root Cause

The CI pipeline failed because of a failing assertion in the test suite `src/tests/failing.test.js`.

The test named `"intentional failure to trigger AI pipeline analysis"` asserts that `2 + 2` should equal `5`. Because the actual evaluated result is `4`, Jest threw an assertion error and exited with status code `1`.

```
expect(2 + 2).toBe(5); // Fails: Expected 5, Received 4
```

2. Affected Job(s)

- Job: `Backend Tests`
- Step: `Run npm test`

3. Fix

Depending on your objective, apply one of the following solutions:

Option A: Correct the math assertion

If you want to keep the test but make it pass, update the expected value in `src/tests/failing.test.js` to `4`:

```
---  
*Analysis generated by [Gemini AI](https://ai.google.dev)*
```

Ai Analysis Failure

Soms is het mogelijk dat het AI model reageert met een foutmelding, bijvoorbeeld als Gemini Down is.



In dit geval was het verkeer bij Gemini erg hoog waardoor er geen comments geplaatst worden door Gemini.

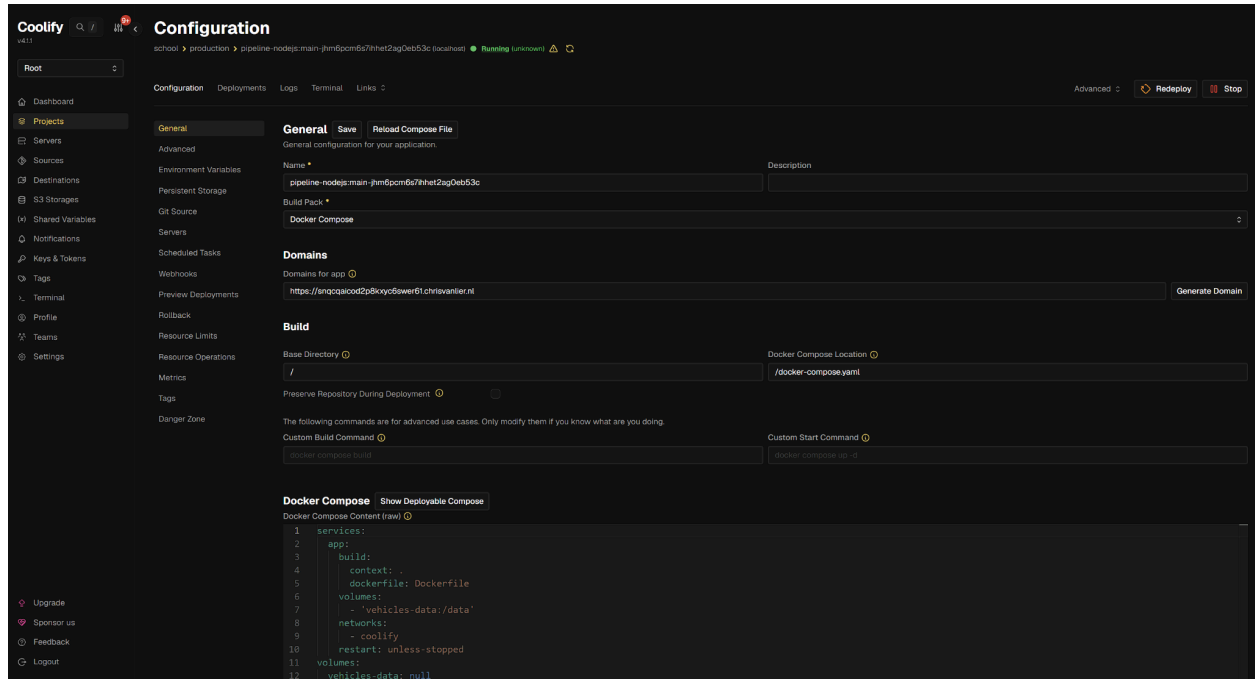
Workflows overzicht

De tijden van de workflows zijn erg gevarieerd, dat hangt af van hoe groot de commits zijn. Over het algemeen is de performance erg goed en snel, maar dat hangt ook af van de AI responstijden bij Gemini omdat we van die API gebruikmaken.

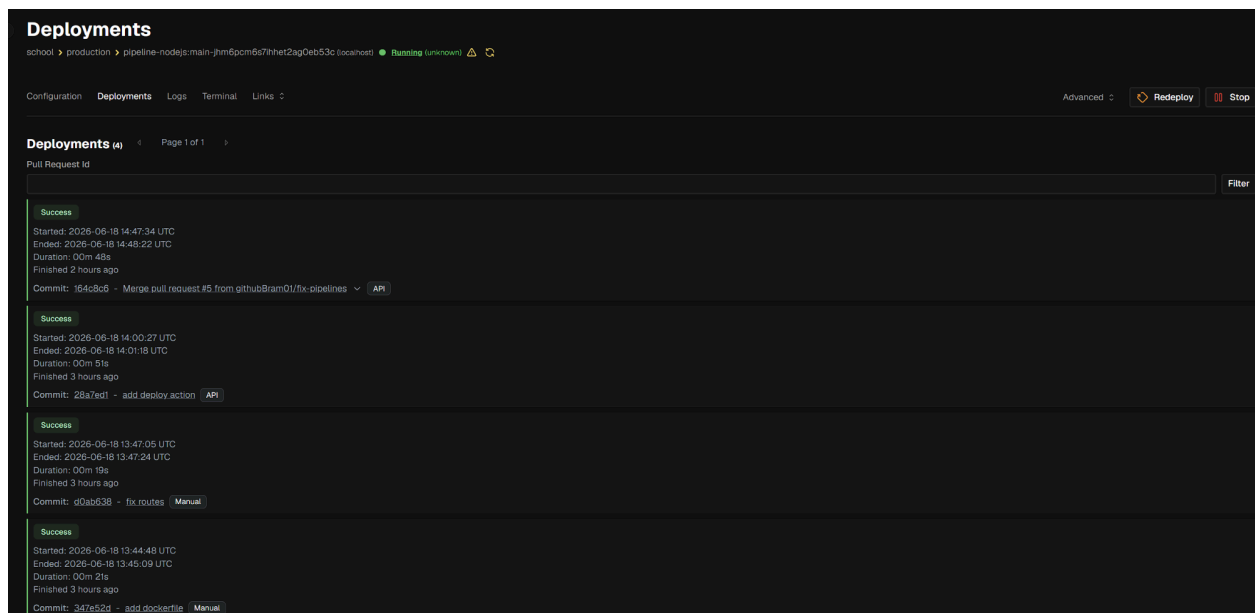
80 workflow runs		Workflow ▾	Event ▾	Status ▾	Branch ▾	Actor ▾
✓	AI Pipeline Failure Analysis	AI Pipeline Failure Analysis #28: completed by chrisvanlier2005		Today at 4:48 PM	...	
				1m 55s		
⌚	AI Pipeline Failure Analysis	AI Pipeline Failure Analysis #27: completed by chrisvanlier2005		Today at 4:47 PM	...	
				2s		
✗	add failing testcase	Vehicle Evaluation CI/CD #28: Pull request #6 opened by chrisvanlier2005		Today at 4:47 PM	...	
		add-failing-test		23s		
⌚	AI Pipeline Failure Analysis	AI Pipeline Failure Analysis #26: completed by chrisvanlier2005		Today at 4:47 PM	...	
				2s		
✓	AI Pipeline Failure Analysis	AI Pipeline Failure Analysis #25: completed by githubBram01		Today at 4:46 PM	...	
				33s		
✓	Merge pull request #5 from githubBram01/fix-pipelines	Vehicle Evaluation CI/CD #27: Commit 164c8c6 pushed by chrisvanlier2005		Today at 4:46 PM	...	
		main		46s		
✓	error	Vehicle Evaluation CI/CD #26: Pull request #5 opened by chrisvanlier2005		Today at 4:46 PM	...	
		fix-pipelines		27s		
✗	Merge pull request #4 from githubBram01/fix-pipelines	Vehicle Evaluation CI/CD #25: Commit 23b4c43 pushed by githubBram01		Today at 4:46 PM	...	
		main		22s		
✓	AI Pipeline Failure Analysis	AI Pipeline Failure Analysis #24: completed by chrisvanlier2005		Today at 4:19 PM	...	
				35s		
✗	Update pipelines	Vehicle Evaluation CI/CD #24: Pull request #4 synchronize by chrisvanlier2005		Today at 4:18 PM	...	
		fix-pipelines		24s		

Coolify Deployment

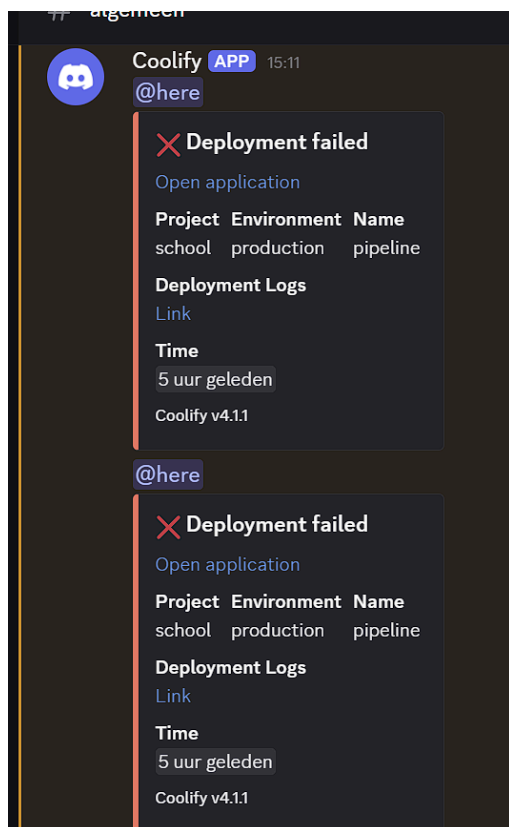
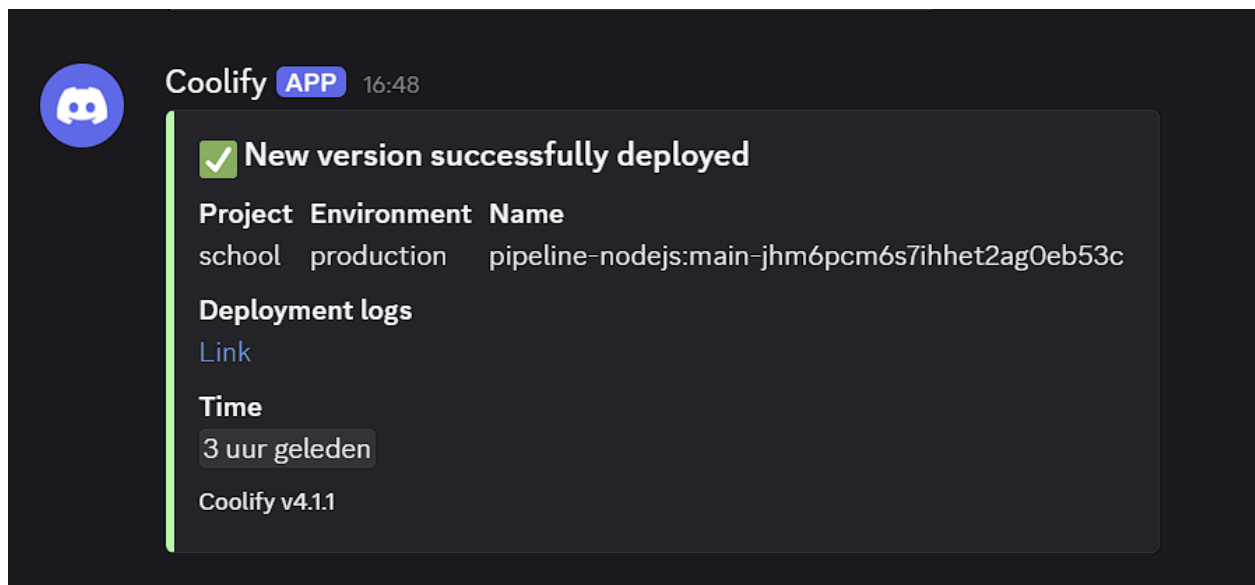
Hieronder is het configuratiescherm binnen coolify. Naast wat instellingen met volumes en een custom domein, staat hier geen custom code in.



Hieronder is te zien dat er verschillende deployments uitgevoerd worden, door zowel de developers handmatig, als de webhook via de API, met de actions.



Hieronder is te zien, dat wij, als developers, een chat bericht krijgen in Discord (chat app), zodra een deployment succesvol is, of onsuccesvol.



4. Evaluatie, Reflectie en Schaalbaarheid

Met de succesvolle implementatie van het prototype is de transitie van CI naar een volwaardige CI/CD-pipeline met geïntegreerde AI-optimalisatie een feit.

Prestatieanalyse: wat heeft AI gebracht?

De introductie van Docker en Coolify zorgt voor een volledig Continuous Delivery (CD)-systeem. Code die door de teststraten komt, staat binnen een minuut in een geïsoleerde productieomgeving, zonder risico op menselijke installatiefouten.

De grootste meerwaarde van de AI-integratie (Gemini) is meetbaar in tijd en cognitieve belasting. Vóór de implementatie moest een developer bij een falende build handmatig door honderden regels ruwe logoutput zoeken om de oorzaak te lokaliseren — een proces dat gemiddeld vijf tot vijftien minuten per fout kon kosten. Met de AI-analyse verschijnt er direct op de pull request een gerichte samenvatting én een concrete codesuggestie. Dat reduceert de diagnosetijd naar seconden en verlaagt de drempel voor minder ervaren developers om zelfstandig fouten op te lossen.

Vergelijking met de oorspronkelijke werkwijze

Aspect	Vóór (CI only)	Na (CI/CD + AI)
Deployment	Handmatig, foutgevoelig	Geautomatiseerd via Coolify
Foutdiagnose	Handmatig logbestanden doorzoeken	AI-samenvatting op de PR
Feedbacksnelheid	Minuten tot uren	Seconden
Vereiste expertise	Hoog (log-interpretatie)	Lager (AI begeleidt)

De voornaamste optimalisatie mogelijkheid ligt in het sluiten van de feedbacklus: momenteel stopt de pipeline bij een fout, maar er is nog geen automatische rollback als een container ná deployment runtime-fouten vertoont. Daarnaast is de log-parsing nog niet helemaal waterdicht: het script knipt af op de laatste 6000 karakters, waardoor bij complexe fouten cruciale informatie verloren kan gaan. Een slimme algoritme dat gericht zoekt op **Error**- en **Stacktrace**-patronen verhoogt zowel de AI-effectiviteit als reduceert het tokenverbruik en

daarmee de API-kosten.

Toepasbaarheid in de praktijk

De oplossing is direct bruikbaar **onder de volgende voorwaarden**:

- **Teamgrootte**: het systeem levert de grootste waarde bij teams van twee of meer developers, waar code-reviews en gedeelde pipelines de norm zijn. In een solo-project is de overhead relatief hoog.
- **Infrastructuur**: de oplossing vereist een server of VPS waarop Coolify draait. Voor organisaties die al cloudinfrastructuur gebruiken (AWS, Azure, GCP) is integratie eenvoudig; voor teams zonder eigen server is een hosted alternatief noodzakelijk.
- **API-kosten**: het Gemini AI-model genereert kosten per API-aanroep. Bij een hoog build-volume (tientallen builds per dag) moeten token-limieten en budgetplafonds worden ingesteld om onverwachte kosten te voorkomen.
- **Gevoeligheid van logdata**: logbestanden kunnen gevoelige informatie bevatten (verbindingstrings, stacktraces met interne paden). Voordat de oplossing in een productieomgeving wordt ingezet, moet worden beoordeeld of het doorsturen van logs naar een externe AI-API voldoet aan het privacybeleid en eventuele AVG-vereisten van de organisatie.

Verbeterpunten en Toekomstvisie (Schaalbaarheid)

Om dit prototype door te ontwikkelen naar een enterprise-ready oplossing, worden de volgende twee verbeterpunten voorgesteld:

- **Automatische Rollbacks (Predictive & Reactive Delivery)**: Momenteel stopt de pipeline bij een fout en wordt er geen slechte code naar Coolify gestuurd. Echter, als een container na een schijnbaar succesvolle deployment runtime-fouten vertoont (health check faalt op de server), moet de pipeline worden uitgebreid om automatisch de voorgaande Docker-image tag terug te zetten. Dit kan worden aangestuurd door Coolify-statusrapportages terug te koppelen naar GitHub.
- **Contextbewuste Kosten- en Token-optimalisatie**: Het huidige script snijdt logbestanden rigide af op de laatste 6000 karakters. Bij complexe of meervoudige job-fouten kan cruciale informatie buiten de scope vallen. Een slimmer log-parsing algoritme dat specifiek zoekt naar 'Error'- en 'Stacktrace'-patronen zal de effectiviteit van de AI verhogen en het tokenverbruik (en daarmee de API-kosten) verder reduceren.

Conclusie

Het prototype bewijst dat de synergie tussen lichtgewicht container-orchestratie (Coolify) en generatieve AI-foutanalyse (Gemini) de ontwikkelsnelheid en stabiliteit van het Voertuig Evaluatieplatform aantoonbaar verhoogt. De twee geïdentificeerde verbeterpunten, automatische rollbacks en contextbewuste log-parsing zouden de enterprise ready route kunnen vormen.